Lecture 0: Introduction

CSI2110 – Data Structures and Algorithms

Lecture study notes

Source document: `../lec00-introduction.pdf`. These notes synthesize the source and add worked explanations. Page references use PDF page numbers, which can differ from printed slide numbers. Pseudocode is normalized for readability and boundary cases.

Contents

1 What this course studies **2**

2 Arrays **2**

2.1 Access and updates 2

3 Singly linked lists **2**

3.1 Local insertion and deletion 3

3.2 What a tail reference does 3

4 Doubly linked lists **3**

5 Choosing an implementation **3**

6 Course logistics recorded in the slides **4**

7 Study checks **4**

1 What this course studies

Source: PDF pages 2–13. An **algorithm** describes steps for solving a problem; a **data structure** stores and organizes data so the required operations can run efficiently. CSI2110 focuses on choosing structures, implementing algorithms in Java, and explaining their time and space costs mathematically.

A phone book stores names and phone numbers and supports insert, delete, and lookup. An unordered list, ordered list, or tree can represent the same information while giving different operation costs. A map of buildings and distances naturally uses a graph and supports questions such as shortest routes.

Key idea. Distinguish logical organization from physical implementation. A list, tree, graph, set, or map describes relationships; arrays, linked nodes, and adjacency lists describe ways to store those relationships.

2 Arrays

Source: PDF pages 29–33. An array is an indexed sequence with a fixed capacity. If the capacity is N, legal indices are $0, \ldots, N-1$. An array of primitives stores primitive values; an array of objects stores references. Java initializes an `int[]` with zeroes and an object-reference array with `null`.

```
int[] x = {3, 1, 7, 5};
int[] y = new int[4];  // {0, 0, 0, 0}
```

Size versus capacity: n is the number of logically stored entries; N is the number of allocated slots. They need not be equal.

2.1 Access and updates

- Reading or replacing the value at a known index uses direct access: $O(1)$.
- To insert at index i in an ordered sequence, shift entries $i, \ldots, n-1$ one slot right. Shift from right to left so values are not overwritten.
- To remove index i, shift entries $i+1, \ldots, n-1$ one slot left.
- If an insertion would exceed capacity, allocate a larger array and copy existing entries. A resize takes $\Theta(n)$ time.

Example: starting with $[3, 1, 5]$, inserting 7 at index 2 produces $[3, 1, 7, 5]$ after shifting 5. Removing index 1 then gives $[3, 7, 5]$.

3 Singly linked lists

Source: PDF pages 34–38,43. Each node stores an element and a reference to the next node. The head refers to the first node; the last node has a null next reference. Nodes need not occupy consecutive memory locations.

```
head -> [3 | next] -> [1 | next] -> [7 | next] -> [5 | null]
```

```
class Node<E> {
    E element;
    Node<E> next;
    Node(E e, Node<E> n) { element = e; next = n; }
}
```

Generics allow the same structure to hold different reference types, such as `Integer` or `String`. Java generics use wrapper types rather than primitives.

3.1 Local insertion and deletion

Given a predecessor `pred`, insert immediately after it:

```
newNode = new Node(e, pred.next)
pred.next = newNode
```

The order matters: save the successor before changing the predecessor's next reference.

Given the predecessor of a node to delete:

```
pred.next = pred.next.next
```

Both changes take $O(1)$ once the predecessor is known. Finding the node at index i requires traversal and takes $O(i+1)$, hence $O(n)$ in the worst case. Removing the head is a separate case: `head = head.next`.

3.2 What a tail reference does

A tail reference permits appending in $O(1)$: link the old tail to the new node and update the tail. Without a tail, appending requires traversal. A tail reference alone does *not* permit constant-time removal of the tail, because its predecessor is not directly accessible. That operation still takes $O(n)$.

When the list becomes empty, both head and tail must be null. When inserting into an empty list, both must refer to the new node. A cached size field permits $O(1)$ `size()`.

4 Doubly linked lists

Source: PDF pages 39–42. A doubly linked node stores an element, a previous reference, and a next reference. With references to both ends, traversal is possible in both directions.

```
null <- [3] <-> [1] <-> [7] <-> [5] -> null
```

To insert e between adjacent nodes `pred` and `succ`:

```
n = new Node(pred, e, succ)
pred.next = n
succ.prev = n
```

To delete an interior node `x`:

```
x.prev.next = x.next
x.next.prev = x.prev
```

Local updates take $O(1)$ when the relevant nodes are known. Index-based lookup still takes $O(n)$ in the worst case. Compared with a singly linked list, each node uses an extra reference.

Sentinels: a header before the first element and a trailer after the last element simplify updates. They store no user elements. An empty sentinel list has `header.next = trailer` and `trailer.prev = header`.

5 Choosing an implementation

Source: PDF pages 44.

Operation	Array sequence	Singly linked*	Doubly linked*
Access by index	$O(1)$	$O(n)$	$O(n)$
Insert/remove in middle	$O(n)$	$O(1)$ local	$O(1)$ local
Append at end	$O(1)$ if room	$O(1)$	$O(1)$
Remove last	$O(1)$	$O(n)$	$O(1)$
Storage	$O(N)$	$O(n)$	$O(n)$

*Assume a tail reference; singly linked local deletion requires the predecessor. An array resize costs $O(n)$. Lookup costs must be added to any local pointer change.

6 Course logistics recorded in the slides

Source: PDF pages 16–26. This section records the supplied Fall 2026 slides; consult Brightspace for subsequent changes.

- Two lectures, one lab, and one tutorial per week. Lectures develop theory; labs practice Java; tutorials practice written reasoning.
- Assessment weights: written assignments 15%, programming assignments 15%, midterm 30%, final 40%, plus up to 2% lab bonus.
- The slides list the midterm on November 7, 9:30–11:30 a.m.; programming assignments in October and November.
- Let A, P, M, F, B be weighted percentage-point contributions. If $(M + F)/70 \geq 0.5$, the stated grade is $A + P + M + F + B$; otherwise it is $100(M + F)/70$.
- Assignments are individual and submitted through Brightspace. The stated late penalty is 30% within 24 hours; submissions after 24 hours are not accepted.
- Exams are closed-book. At least 50% on the weighted exam component is needed to pass. An approved missed midterm shifts its weight to the final.
- Labs are released Thursday at 4 p.m., due the following Thursday at 11:59 p.m., and require in-person TA grading within two weeks of the deadline. Session-specific details are on Brightspace.
- Instructor office hours in the slides: Tuesday 3–4 p.m. and Thursday 2–3 p.m., SITE 4021. TA office hours are on Brightspace; email replies may take two business days.
- The slides prohibit generative AI in submissions and require references for reused Internet code. These documents are study notes.

7 Study checks

1. Why is array access $O(1)$ but linked-list access by index $O(n)$? **Answer:** an index directly identifies an array slot; a list requires following links.
2. Is linked-list insertion always $O(1)$? **Answer:** only the pointer update is; locating an insertion position can take $O(n)$.
3. Why does a doubly linked list make tail deletion fast? **Answer:** `tail.prev` directly identifies the new tail.
4. What happens when an array is full? **Answer:** allocate more space and copy, or reject insertion in a fixed-capacity implementation.